

IFN647 Week 4 review questions

SOLUTIONS

Q1 Which of the following is wrong? and justify your answer.

- (1) The index is inverted because usually we think of words being a part of documents, but if we invert this idea, documents are associated with words.
- (2) In an inverted index that contains only document information, i.e., the features are binary, meaning they are 1 if the document contains a term, 0 otherwise. This inverted index contains enough information to tell if the document contains the exact phrase “tropical fish”.
- (3) Each index term has its own inverted list that holds the relevant data for that term. Each list entry is called a posting, and the part of the posting that refers to a specific document or location is often called a pointer.
- (4) An extent is a contiguous region of a document. We can represent these extents using word positions.

Solution: (2)

Although a document that contains the words “tropical” and “fish” is likely to be relevant; however, we really want to know if the document contains the exact phrase “tropical fish”. To do this, we need to add position information to the index.

Q2 (Query processing)

Inverted indexing is an efficient data structure to represent documents for information retrieval, where each index term is associated with an inverted list that contains a list of pairs of document number and count of the term occurrences in that document. The following table is an inverted index for 4 documents and their index terms.

and	[1:1]	only	[2:1]
aquarium	[3:1]	pigmented	[4:1]
are	[3:1] [4:1]	popular	[3:1]
around	[1:1]	refer	[2:1]
as	[2:1]	referred	[2:1]
both	[1:1]	requiring	[2:1]
bright	[3:1]	salt	[1:1] [4:1]
coloration	[3:1] [4:1]	saltwater	[2:1]
derives	[4:1]	species	[1:1]
due	[3:1]	term	[2:1]
environments	[1:1]	the	[1:1] [2:1]
fish	[1:2] [2:3] [3:2] [4:2]	their	[3:1]
fishkeepers	[2:1]	this	[4:1]
found	[1:1]	those	[2:1]
fresh	[2:1]	to	[2:2] [3:1]
freshwater	[1:1] [4:1]	tropical	[1:2] [2:2] [3:1]
from	[4:1]	typically	[4:1]
generally	[4:1]	use	[2:1]
in	[1:1] [4:1]	water	[1:1] [2:1] [4:1]
include	[1:1]	while	[4:1]
including	[1:1]	with	[2:1]
iridescence	[4:1]	world	[1:1]
marine	[2:1]		
often	[2:1] [3:1]		

Assume the abstract model of ranking is

$$R(Q, D) = \sum_i g_i(Q) f_i(D)$$

where f_i is the document topical feature function (the value of $f_i(D)$ is term t_i 's counts in document D) and g_i is a query topical feature function ($g_i(Q) = 1$ if term t_i is in query Q ; otherwise, $g_i(Q) = 0$). Let query $Q = \{\text{freshwater, fish, aquarium, tropical}\}$. Calculate each document's ranking score by using Term-at-a-time Algorithm. You are required to write the calculating process (or steps).

Solution:

L1 - freshwater	1:1	4:1		
L2 - aquarium	3:1			
L3 - fish	1:2	2:3	3:2	4:2
L4 - tropical	1:2	2:2	3:1	
L1 -	1:1		4:1	
Partial scores	1:1		4:1	

Old Partial scores	1:1			4:1
L2 -		3:1		
new Partial scores	1:1		3:1	4:1

Old Partial scores	1:1		3:1	4:1
L3 -	1:2	2:3	3:2	4:2
new Partial scores	1:3	2:3	3:3	4:3

Old Partial scores	1:3	2:3	3:3	4:3
L4 -	1:2	2:2	3:1	
final scores	1:5	2:5	3:4	4:3

Q3 (Markov chain)

Assume when John is sad today, which isn't very usual: he either goes for a run, gobbles down ice cream or takes a nap next day. The Markov Chain depicted in the following **state diagram** has 3 possible states: "Sleep", "Run", and "Ice Cream".

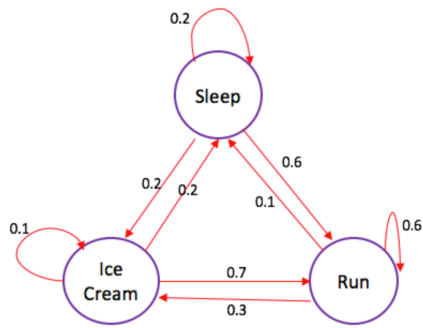


Fig 2. Markov chain example.

The following is the corresponding **transition matrix**

CURRENT STATE	NEXT STATE		
	SLEEP	RUN	ICE CREAM
SLEEP	0.2	0.6	0.2
RUN	0.1	0.6	0.3
ICE CREAM	0.2	0.7	0.1

- (1) According to the above diagram, if John spent sleeping a sad day away, what is the probability of he will likely go for a run next day?
- (2) The transition matrix will be a 3×3 matrix. To simple represent the matrix, we can use the following variables, for example,

SS – Sleep $\rightarrow$ Sleep
 SR – Sleep $\rightarrow$ Run
 SI – Sleep $\rightarrow$ Ice Cream

...

Then the transition matrix for the states is represented as a list of lists in python
`[["SS","SR","SI"], ["RS","RR","RI"], ["IS","IR","II"]]`

Please write the corresponding transition matrix for probabilities in a list of lists.

Solutions:

- (1) It is 60%
- (2) `[[0.2,0.6,0.2], [0.1,0.6,0.3], [0.2,0.7,0.1]]`

Q4 (Optional) Markov chain cont.

We can represent the transition matrix of Q3 as follows:

```
import numpy as np
import random as rm
```

```
# The statespace
states = ["Sleep", "Icecream", "Run"]
# Possible sequences of events
transitionName = [{"SS", "SR", "SI"}, {"RS", "RR", "RI"}, {"IS", "IR", "II"}]
# Probabilities matrix (transition matrix)
transitionMatrix = [[0.2, 0.6, 0.2], [0.1, 0.6, 0.3], [0.2, 0.7, 0.1]]
```

```
if sum(transitionMatrix[0]) + sum(transitionMatrix[1]) + sum(transitionMatrix[2]) != 3:
    print("Somewhere, something went wrong. Transition matrix, perhaps?")
else: print("All is gonna be okay, you should move on!! ;)")
```

All is gonna be okay, you should move on!! ;)

Design a function to forecast the system state after k days. For example, you may assume that the initial state is "Sleep". You may use “numpy.random.choice” to randomly sample a transition from the set of possible state transitions for each day. Your function prints the results in the same format as the sample output shown below.

```
Start state: Sleep
Possible states: ['Sleep', 'Run', 'Run']
End state after 2 days: Run
Probability of the possible sequence of states: 0.3
```

Solution:

```
# A function that implements the Markov model to forecast the state/mood.
# You will use the numpy.random.choice to generate a random sample from the set of transitions possible.
def activity_forecast(days):
    # Choose the starting state
    activityToday = "Sleep"
    print("Start state: " + activityToday)
    # Shall store the sequence of states taken. So, this only has the starting state for now.
    activityList = [activityToday]
    i = 0
    # To calculate the probability of the activityList
    prob = 1
    while i != days:
        if activityToday == "Sleep":
            change = np.random.choice(transitionName[0], replace=True, p=transitionMatrix[0])
            if change == "SS":
                prob = prob * 0.2
                activityList.append("Sleep")
            pass
            elif change == "SR":
                prob = prob * 0.6
                activityToday = "Run"
                activityList.append("Run")
            else:
                prob = prob * 0.2
                activityToday = "Icecream"
                activityList.append("Icecream")
        elif activityToday == "Run":
            change = np.random.choice(transitionName[1], replace=True, p=transitionMatrix[1])
            if change == "RR":
                prob = prob * 0.5
                activityList.append("Run")
            pass
            elif change == "RS":
                prob = prob * 0.2
                activityToday = "Sleep"
                activityList.append("Sleep")
            else:
                prob = prob * 0.3
                activityToday = "Icecream"
                activityList.append("Icecream")
        elif activityToday == "Icecream":
            change = np.random.choice(transitionName[2], replace=True, p=transitionMatrix[2])
            if change == "II":
                prob = prob * 0.1
                activityList.append("Icecream")
            pass
            elif change == "IS":
                prob = prob * 0.2
                activityToday = "Sleep"
                activityList.append("Sleep")
            else:
                prob = prob * 0.7
                activityToday = "Run"
                activityList.append("Run")
        i += 1
    print("Possible states: " + str(activityList))
    print("End state after " + str(days) + " days: " + activityToday)
    print("Probability of the possible sequence of states: " + str(prob))

# Function that forecasts the possible state for the next 2 days
activity_forecast(2)
```

```
Start state: Sleep
Possible states: ['Sleep', 'Run', 'Run']
End state after 2 days: Run
Probability of the possible sequence of states: 0.3
```

Q5 (LLM-based Information Extraction)

Given the following XML document:

```
<newsitem itemid="780718" id="root" date="1997-08-06" xml:lang="en">
<title>EU: Official Journal contents - OJ C 237 of August 4, 1997.</title>
<headline>Official Journal contents - OJ C 237 of August 4, 1997.</headline>
<text>
<p>*</p>
<p>Common position (EC) No 28/97 of 17 June 1997 adopted by the Council, acting in
accordance with the procedure referred to in Article 189b of the Treaty establishing
the European Community, with a view to adopting a European Parliament and Council
Directive on the legal protection of designs</p>
<p>Common position (EC) No 29/97 of 17 June 1997 adopted by the Council, acting in
accordance with the procedure referred to in Article 189b of the Treaty establishing
the European Community, with a view to adopting a European Parliament and Council
Directive amending Directives 76/116/EEC, 80/876/EEC, 89/284/EEC and 89/530/EEC on the
approximation of the laws of the Member States relating to fertilizers END OF
DOCUMENT.</p>
</text>
<copyright>(c) Reuters Limited 1997</copyright>
</newsitem>
```

Design an LLM-based information extraction prompt to extract entities “Itemid”, “Organization”, “Date” and “Copyright” from the XML document, and return the result in a valid Python dictionary as follows:

```
{
  "Itemid": 780718,
  "Organization": "Reuters Limited",
  "Location": "European Community",
  "Date": "1997-08-06",
  "Copyright": "(c) Reuters Limited 1997"
}
```

Solution:

You are an information extraction system. Extract entities from the XML document, and return the result strictly in valid Python dictionary using this template: { Itemid: [integer], Organization: [string], Location: [string], Date: [string], Copyright: [string] }

XML document:

```
<newsitem itemid="780718" id="root" date="1997-08-06" xml:lang="en">
<title>EU: Official Journal contents - OJ C 237 of August 4, 1997.</title>
<headline>Official Journal contents - OJ C 237 of August 4, 1997.</headline>
<text> <p>*</p> <p>Common position (EC) No 28/97 of 17 June 1997 adopted by the Council, acting in
accordance with the procedure referred to in Article 189b of the Treaty establishing the European Community, with
a view to adopting a European Parliament and Council Directive on the legal protection of designs</p>
<p>Common position (EC) No 29/97 of 17 June 1997 adopted by the Council, acting in accordance with the
procedure referred to in Article 189b of the Treaty establishing the European Community, with a view to adopting a
European Parliament and Council Directive amending Directives 76/116/EEC, 80/876/EEC, 89/284/EEC and
89/530/EEC on the approximation of the laws of the Member States relating to fertilizers END OF
DOCUMENT.</p> </text>
<copyright>(c) Reuters Limited 1997</copyright>
</newsitem>
```